GEAR-Nav: Geometric Energy-Guided Aggregation and Refinement for Vision-Language Navigation

anonymous

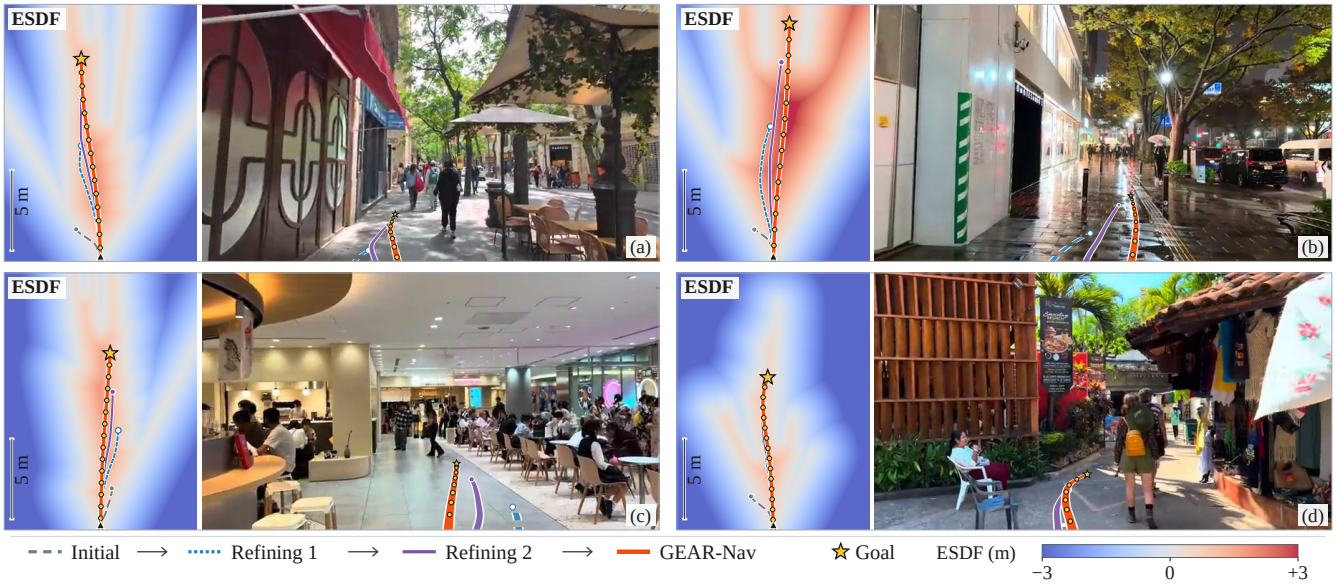


Fig. 1. **Learning to refine paths from geometric energy.** GEAR learns a trajectory-correction field from energy-derived supervision, then repeatedly refines a candidate path using the image, goal, and current path. Across (a) a café sidewalk, (b) a rainy night sidewalk, (c) an indoor dining concourse, and (d) a market passage, conceptual stages extend and reshape short sampled priors toward the goal. Later paths are authored to illustrate progression observed in a recorded rollout; stage numbers are not update counts. Each ESDF and uncropped camera view shows the same metric paths through the saved camera projection. ESDFs visualize training geometry; inference requires neither the distance field nor the energy function.

***Abstract*— Vision-Language-Action (VLA) models provide a promising framework for goal-conditioned robot navigation, but training obstacle-aware policies typically relies on target trajectories obtained from robot demonstrations or offline trajectory optimization. We introduce GEAR-Nav (GEAR), a framework for training navigation VLAs from in-the-wild egocentric video without target trajectory supervision. Given an RGB observation and navigation goal, we reconstruct local scene geometry and define an analytic energy over candidate waypoint trajectories that captures goal reaching, collision avoidance, progress, and motion smoothness. Rather than solving this energy to produce a trajectory label, we use its local descent direction as privileged supervision for learning an autonomous trajectory-refinement field. Starting from simple, kinematically feasible trajectory priors, the learned field iteratively improves candidate paths. To address the distribution shift between initial priors and model-induced trajectories, we introduce an energy-oracle dataset aggregation procedure that repeatedly queries projection-aware energy-descent directions at trajectories visited by the current model, aggregates these state-direction pairs, and trains the VLA on the resulting replay distribution. Geometry and the analytic energy are used only during training; at inference time, the model iteratively refines trajectories using visual observations, goal conditioning, and kinematic projection alone, without access to a distance field, energy function, or online trajectory optimizer. We evaluate our approach on held-out egocentric scenes and real-world robot navigation tasks, measuring goal reaching, collision avoidance, obstacle clearance, and trajectory progress. Our results show that [RESULTS]. These findings demonstrate that geometric energy can directly supervise navigation VLAs, eliminating the need to first convert scene geometry into planner-generated target trajectories.**

I. INTRODUCTION

Vision-Language-Action (VLA) models connect visual observations and language instructions to robot actions, bringing semantic knowledge from large-scale pretraining into embodied control [**?**]. Navigation VLAs extend this interface to destinations specified by language, images, or spatial coordinates [4], [2]. Scaling these capabilities to everyday mobile robots, however, requires supervision that covers diverse environments, goals, and obstacle configurations. Collecting robot demonstrations across these combinations is costly, particularly in cluttered indoor and outdoor spaces where recognizing a destination must be coupled with close-range geometric reasoning.

The challenge is not simply to record more motion. A traversal shows how to move toward one destination, while the same observation may support many other goals, each requiring a different route through the available free space. Reaching a doorway, approaching an object, or passing around furniture can demand different choices even from

the same starting viewpoint. A broadly goal-conditioned policy therefore needs supervision that relates the requested destination to obstacle clearance, traversability, and progress, rather than relying only on the path that happened to be demonstrated. Geometric planning objectives express these requirements explicitly; transferring them into a visual policy offers a way to teach obstacle-aware behavior without requiring online geometric mapping at deployment.

In-the-wild egocentric videos offer a scalable source of observations for this purpose. Walking tours and other first-person recordings capture varied layouts, viewpoints, and obstacles without dedicated robot data collection. Yet these videos do not directly provide robot-executable paths toward every potential goal in a scene. Existing approaches recover motion from video [**?**] or use a pretrained navigation policy to supply trajectory supervision [**?**]. Reconstructed geometry provides another route: VEGA [1] generates obstacle-aware reference paths toward scene-derived goals and uses them to train a flow-matching navigation VLA. In each case, learning relies on a target trajectory, whether observed, predicted, or planned. Geometry, however, contains more than a preferred final path: it also indicates how a candidate path should change to improve clearance and goal-directed progress.

This raises the question: *can geometric objectives teach a navigation VLA to refine trajectories without first producing target paths?* Learning navigation directly from differentiable costs is established by methods such as iPlanner and ViPlanner [**?**], [**?**]. Our focus is on learning local corrections conditioned on the image, goal, and current candidate trajectory, so that the same model can be applied repeatedly as that trajectory evolves. This requires both a geometric supervision signal that respects the constraints on path updates and training coverage of the candidate paths encountered during learned refinement.

We introduce **GEAR-Nav** (GEAR), *Geometric Energy-Guided Aggregation and Refinement for Vision-Language Navigation*. GEAR adapts a pretrained VLA using egocentric observations and goals expressed through language, image regions, or robot-frame waypoints. During training, reconstructed scene geometry defines an analytic energy over candidate paths that encodes collision avoidance, obstacle clearance, goal reaching, progress, and smooth motion. Instead of minimizing this energy to create a reference trajectory, we query its local descent direction. A projection-aware oracle scales this correction through backtracking on projected proposals, accounting for the fixed origin and bounded waypoint segments. The VLA learns to predict the resulting correction from the observation, goal, and candidate path. Its field is autonomous: the update rule depends on the current path, without a refinement-time input or a noise-to-target interpolation.

Repeated prediction creates a second challenge: each learned correction changes the candidate supplied to the next update. Training only on initial paths leaves these later inputs outside the supervised distribution. Inspired by dataset aggregation [**?**], GEAR starts from goal-biased unicycle priors, advances candidate paths with the current learned field, queries oracle corrections at the resulting states, and retains these examples across training rounds. An energy acceptance guard controls frontier updates during collection. At deployment, both the guard and the oracle are removed: refinement uses only the RGB observation, goal, candidate path, and lightweight kinematic projection. Neither an ESDF nor analytic energy evaluations or gradients are required at inference. The absence of target trajectories applies to this navigation adaptation; it does not imply action-free pretraining of the VLA backbone.

Our contributions are threefold:

- **Local geometric supervision for navigation VLAs:** We formulate navigation adaptation using energy-derived trajectory corrections, without demonstrated or planner-generated target paths.
- **Projection-aware supervision and trajectory-space aggregation:** We construct local correction labels consistent with projected updates and aggregate them at model-induced candidate paths to train repeated refinement beyond the initial prior distribution.
- **Autonomous visual trajectory refinement:** We adapt a pretrained VLA into a reusable, multimodal goal-conditioned correction field that refines candidate paths without the training-time geometry, analytic energy, or acceptance guard at deployment.

II. RELATED WORK

A. Navigation VLAs and Learning from Video

Navigation VLAs connect visual-language representations with embodied control. NaVILA [4] uses a hierarchical architecture in which a VLA generates intermediate spatial commands for a visual locomotion policy. OmniVLA [2] supports language, image, and spatial goals, including combinations of modalities. These approaches establish useful interfaces for goal-conditioned navigation; GEAR focuses on the supervision used to learn obstacle-aware waypoint updates within such an interface.

Egocentric video provides an alternative to collecting task-specific robot demonstrations. CityWalker [**?**] extracts motion supervision from web-scale walking and driving videos using visual odometry. LeLaN [**?**] combines semantic goal annotation and goal-reaching objectives with trajectory-based distillation from a pretrained navigation model to teach obstacle avoidance. VEGA [1] instead reconstructs local geometry and generates MPPI trajectories toward multiple multimodal goals, using these paths to supervise a flow-matching VLA. GEAR shares VEGA's use of video-derived geometry and aligned goals, but changes the supervision: it queries local corrections at candidate paths instead of constructing optimized target trajectories. Thus, its distinction is not the use of action-free video or multimodal goals alone, but how geometry supervises the navigation model.

B. Cost-Supervised Planning and Learned Optimization

Trajectory optimization methods such as CHOMP [**?**] improve initial paths using geometric gradients and smoothness objectives. Their update rules provide a classical foundation

for iterative trajectory refinement, but require access to the planning objective when solving a new problem. Learning-based approaches can transfer these objectives into a policy. iPlanner [**?**] trains a navigation network through differentiable planning costs without labeled trajectories, while ViPlanner [**?**] extends imperative learning to geometric and semantic traversability. Consequently, learning navigation from costs without target paths is established prior work.

GEAR differs in the quantity it learns: a conditional correction field over candidate trajectories. Analytic gradients are converted into cached, projection-aware direction labels, and network training regresses those labels rather than backpropagating the trajectory energy through the network. This also relates to learned optimization, where parameterized update rules exploit structure shared across optimization problems [**?**]. Unlike the gradient-input recurrent optimizer of Andrychowicz et al., GEAR predicts updates from visual observations, goal conditioning, and the current path; analytic energy gradients are training targets, not deployment inputs.

C. Generative Trajectory Models and Cost Guidance

Diffusion Policy [**?**] models robot actions through conditional denoising, while flow matching [**?**] learns transport fields between source and target distributions. These methods motivate expressive action generators, but their conventional supervised formulations learn from action or trajectory samples. Motion Planning Diffusion [**?**] combines a learned trajectory prior with cost gradients during denoising, allowing generated motions to adapt to new planning constraints. Such cost-guided inference retains explicit access to the task objective.

GEAR also performs iterative updates, but its training targets are local energy corrections rather than denoising targets or noise-to-trajectory displacements. It learns an autonomous refinement field, with no refinement-time conditioning, and removes the analytic energy at deployment. Its objective is to learn repeated trajectory improvement, not to match a prescribed trajectory distribution. This separates GEAR from both trajectory-supervised generative policies and inference-time cost-guided samplers.

D. Dataset Aggregation and Planner Distillation

DAgger [**?**] addresses sequential distribution shift by querying expert labels at states encountered by the learner and aggregating these examples across training iterations. This principle has also been applied to planning: DeepPANTHER [**?**] uses dataset aggregation to train a trajectory-generating policy from an optimization-based expert. These precedents distinguish dataset aggregation itself from the specific oracle and state space used in GEAR.

In GEAR, the aggregated states are candidate waypoint paths associated with fixed observation-goal conditions, and the labels are local energy-descent directions rather than complete expert trajectories. The learned field advances the candidate frontier under a training-only energy guard, and subsequent oracle queries supervise corrections at the new states. The resulting method combines trajectory-space aggregation with projection-aware local supervision, while evaluating the deployed policy without the oracle or acceptance guard.

III. BACKGROUND

A. Goal-Conditioned Navigation VLAs

Let I denote the current RGB observation and g the goal specification, which may be a language instruction, an image region, or a robot-frame coordinate. We represent a local navigation trajectory as

$$x = (p_0, \dots, p_{H-1}) \in \mathbb{R}^{H\times 2}, \qquad p_0 = (0,0), \tag{1}$$

where H is the number of waypoints and all positions are expressed in the robot's local frame. A navigation policy models $\pi_\theta(x \mid I, g)$. Pretrained VLAs provide visual-language context that can condition an action expert, while navigation-specific encoders accommodate spatial goal inputs [**?**], [2]. In this work, *action-free* refers to the task-specific training data: no demonstrated actions or optimized target paths supervise GEAR's navigation adaptation. It does not imply that the pretrained VLA was learned without robot-action data.

B. Flow Matching with Target Trajectories

Flow matching learns a vector field that transports a simple source distribution toward a target data distribution [**?**]. For trajectory prediction, let z_1 collect the non-origin waypoints of a target path and let z_0 be an independent standard Gaussian sample of the same dimension. A common linear interpolation is

$$z_\tau = (1-\tau)z_0 + \tau z_1, \qquad \tau \sim \mathcal{U}[0,1]. \tag{2}$$

The conditional vector field is trained with

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}\Big[\|v_\theta(z_\tau, \tau, I, g) - (z_1 - z_0)\|_F^2\Big], \tag{3}$$

where the expectation is over observation-goal pairs, target trajectories, source noise, and interpolation times. At inference, integrating the learned field from $\tau = 0$ to $\tau = 1$ produces a trajectory sample. This supervised construction requires target trajectory samples. An energy gradient at a candidate path is a different supervision signal; substituting it for $z_1 - z_0$ does not by itself retain the same distribution-matching objective.

C. Geometric Energy and Local Refinement

A local Euclidean Signed Distance Field (ESDF) Φ assigns signed distances to positions in the navigation plane. In the visibility-aware representation used here, positive values indicate observed free space, while occupied and unobserved regions receive non-positive values. Interpolating this field along a candidate path allows geometric preferences to be expressed through a trajectory energy $E(x; g, \Phi)$. For language or image goals, geometric goal terms use the corresponding training-time metric goal annotation; this annotation is not an additional input to the learned field unless waypoint conditioning is explicitly selected.

Where the energy is differentiable and its gradient is nonzero, $-\nabla_x E(x; g, \Phi)$ gives an unconstrained local descent direction. A projected gradient step takes the form

$$x^+ = \Pi_{\mathcal{K}}(x - \alpha \nabla_x E(x; g, \Phi)), \tag{4}$$

where $\alpha > 0$ is a step size and $\Pi_{\mathcal{K}}$ enforces the chosen waypoint constraints, including the fixed origin. Finite steps and constraint handling require care: an arbitrary projected proposal need not decrease the energy. GEAR therefore accounts for the acceptance of projected proposals when constructing oracle labels, as detailed in Sec. IV-C. Waypoint constraints alone do not guarantee obstacle avoidance.

D. Dataset Aggregation for Sequential Prediction

When a predictor's output determines its next input, prediction errors can move subsequent inputs away from the training distribution. DAgger addresses this issue by collecting states encountered by the current policy, obtaining expert labels at those states, and retraining on their union with earlier data [**?**]. In trajectory refinement, the analogous state is a candidate path rather than a newly executed robot state. GEAR applies this principle in trajectory space, using an analytic energy oracle to label corrections. Its aggregation mechanism targets the states encountered during refinement, rather than only the initial trajectory distribution.

IV. METHODOLOGY

We introduce **GEAR**, an approach for training goal-conditioned navigation VLAs without target trajectory supervision. Rather than first solving a trajectory optimization problem and using the resulting trajectory as a training label, GEAR uses reconstructed scene geometry to define an analytic energy over candidate trajectories. The local descent direction of this energy provides privileged supervision for learning an autonomous trajectory-refinement field. Figure 2 summarizes the training pipeline.

Given an RGB observation I, a goal specification g, and a candidate waypoint trajectory

$$x = \{p_h\}_{h=0}^{H-1}, \qquad p_h \in \mathbb{R}^2,$$

the learned field predicts a direction

$$F_\theta(I, g, x) \in \mathbb{R}^{H \times 2}$$

that improves the current trajectory. During training, an ESDF Φ is available as privileged geometric information. At deployment, the ESDF and the analytic energy are discarded, and the learned field is iteratively applied using only the visual observation, goal conditioning, and current candidate trajectory.

A. Action-Free Navigation Conditions

We construct training conditions from in-the-wild egocentric observations. Each condition consists of an RGB image I, a navigation goal g, and a local Euclidean Signed Distance Field (ESDF) Φ reconstructed from scene geometry. Goals can be specified through language, an image region, or a robot-frame waypoint.

Importantly, no reference navigation trajectory is associated with a training condition. In particular, planned trajectories, robot demonstrations, and trajectory actions are not used as supervision. The geometric representation is used only to evaluate candidate trajectories and construct the local energy-descent supervision described below.

For a waypoint p, we define robot-footprint clearance as

$$c(p) = \Phi(p) - r_{\text{robot}},$$

where r_{robot} is the robot radius. Occupied, unknown, invalid, and out-of-map regions are treated conservatively when evaluating candidate trajectories.

B. Geometric Trajectory Energy

We define a differentiable trajectory energy

$$E(x; g, \Phi) = \sum_{q \in \mathcal{Q}} \lambda_q E_q(x; g, \Phi),$$

where the energy terms

$$\mathcal{Q} = \{\text{collision}, \text{clearance}, \text{goal}, \\ \text{progress}, \text{heading}, \text{smoothness}\}$$

capture complementary properties of a desirable navigation trajectory.

The collision term penalizes trajectories that violate robot-footprint clearance or enter unsupported geometric regions. The clearance term can encourage separation from nearby obstacles, while the goal and progress terms encourage the trajectory to move toward the requested target. An early-heading term discourages initially moving away from the goal, and a smoothness term regularizes abrupt changes along the trajectory. Safety-related terms are evaluated on dense samples along waypoint segments rather than only at the predicted waypoints.

Unlike planner-based supervision, we do not minimize E offline to obtain a target path. Instead, the energy defines how the current candidate trajectory should locally change.

C. Projection-Aware Energy Oracle

For a candidate trajectory x_k at aggregation depth k, we query an analytic energy oracle by differentiating the trajectory energy with respect to the trajectory itself:

$$g_k = \nabla_x E(x_k; g, \Phi).$$

We convert the raw gradient into a normalized descent direction. Let $\rho(g_k)$ denote the root-mean-square gradient magnitude over all non-origin waypoints. The initial oracle direction is

$$\tilde{d}_k = -\frac{g_k}{\max(\rho(g_k), \epsilon_{\text{grad}})}.$$

Directions below a minimum gradient magnitude are set to zero, and large directions are capped to prevent unstable trajectory updates.

Candidate updates must additionally satisfy the trajectory parameterization and kinematic constraints. Let $\Pi_{\mathcal{K}}$ denote the projection that keeps the first waypoint fixed at the robot origin and converts the candidate into a bounded, physically

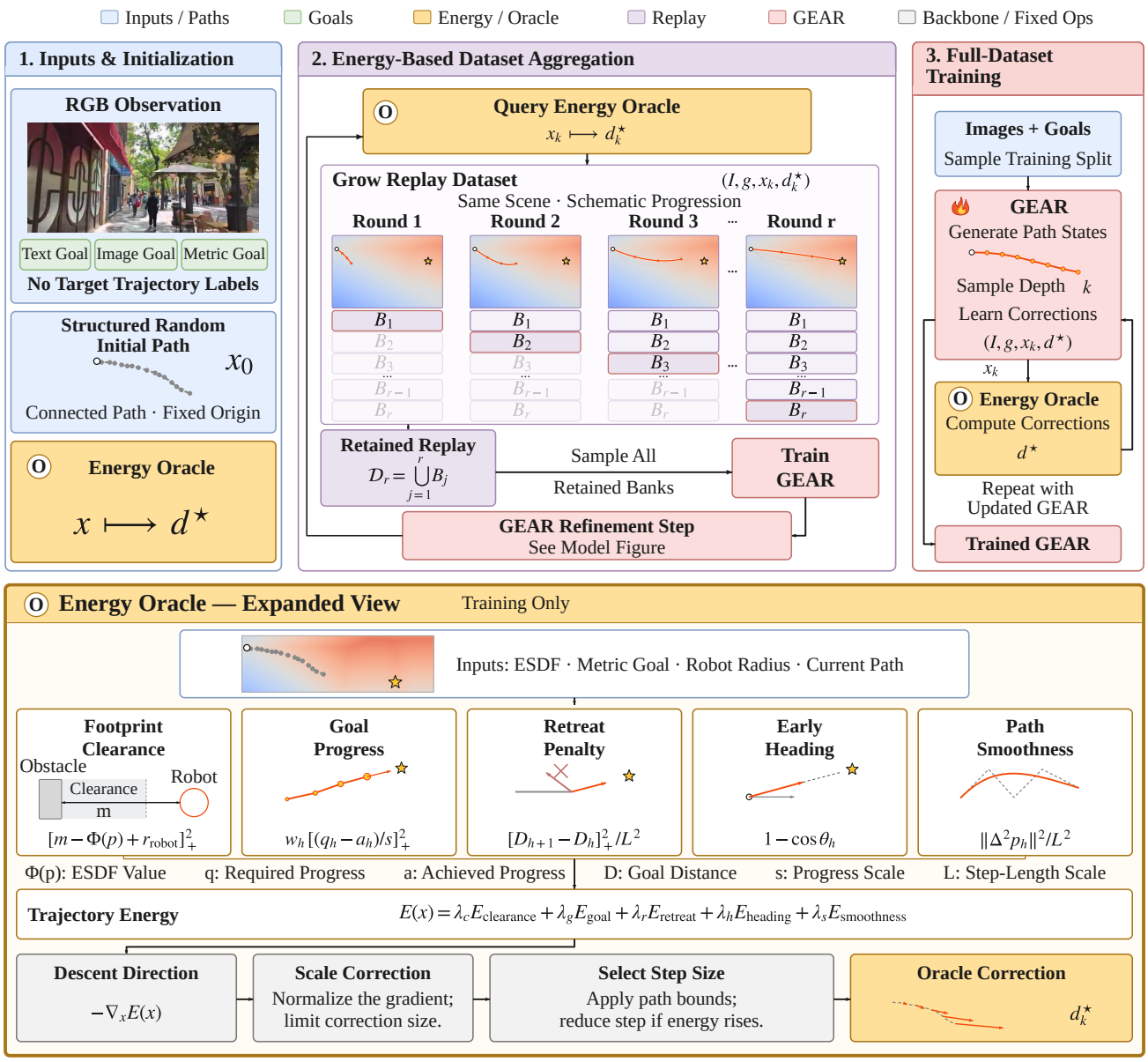


Fig. 2. GEAR training pipeline. Local energy supervision supplies correction targets, which are aggregated over candidate paths before continuation on additional training conditions. In full-dataset training, the current GEAR model generates candidate paths, the energy oracle supplies correction targets, and the same model learns from those targets. The updated model generates paths for the next cycle. The positive-part operator is $[z]_+ = \max(0, z)$.

valid waypoint sequence. Starting from a nominal step size η, we evaluate

$$x'_k = \Pi_{\mathcal{K}}\left(x_k + \alpha \tilde{d}_k\right),$$

and geometrically backtrack α until a finite, non-energy-increasing proposal is found:

$$E(x'_k; g, \Phi) \le E(x_k; g, \Phi) + \epsilon_E.$$

If α_k is the accepted step size, the cached oracle label is the correspondingly scaled pre-projection direction

$$d_k^\star = \frac{\alpha_k}{\eta} \tilde{d}_k.$$

This construction makes the oracle supervision consistent with the projected update that is subsequently applied to candidate trajectories.

D. Autonomous VLA Trajectory Field

Figure 3 shows the correction model and iterative refinement procedure.

GEAR builds on a pretrained $\pi_{0.5}$ VLA backbone. Visual and goal conditioning form the prefix context, while the current candidate waypoint trajectory is provided to the action expert. The action expert is repurposed to predict a trajectory-improvement field

$$F_\theta(I, g, x_k) \approx d_k^\star.$$

In contrast to standard flow-matching training, we do not construct an interpolation between noise and a ground-truth trajectory. The model time token is fixed to a constant value throughout training and inference. Consequently, F_θ

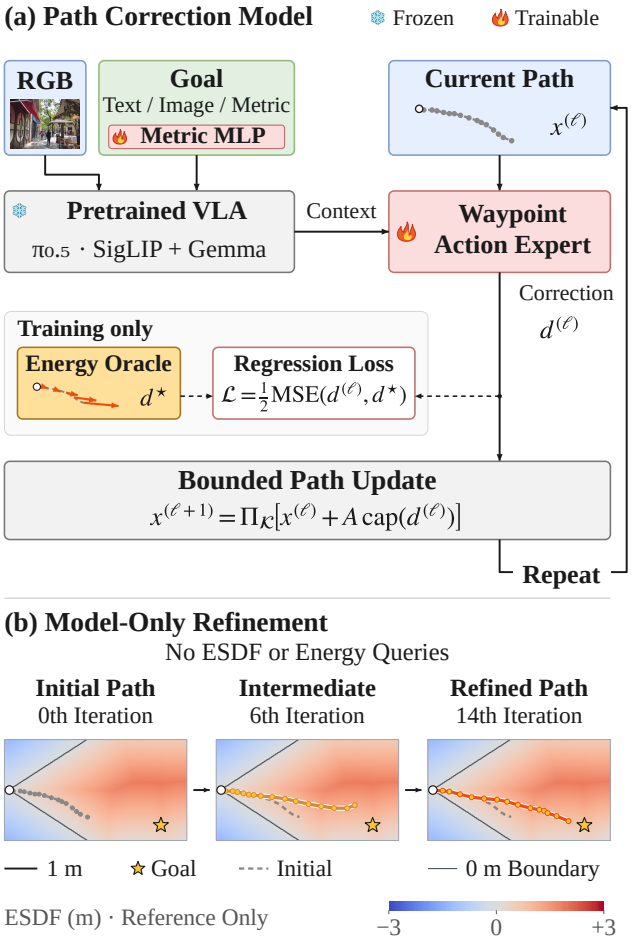


Fig. 3. **GEAR correction model and refinement.** (a) RGB/goal context and the current path enter the Waypoint Action Expert, shown as one block with trainable adapters and correction head that predict $d^{(\ell)}$. Dashed branches regress energy-oracle corrections at aggregated paths, without target trajectories. Half-MSE excludes the fixed origin. The update caps correction magnitude, applies waypoint weights A, and enforces origin/segment bounds through $\Pi_{\mathcal{K}}$; the next path re-enters the same model. (b) Recorded initial, intermediate, and refined paths at a shared metric scale; dashed gray paths show the initial candidate. The ESDF is a visualization reference. Inference uses no ESDF or energy queries.

represents an *autonomous* vector field over trajectory space: its prediction depends on the observation, goal, and current trajectory, but not on aggregation depth k.

For an aggregated training tuple $(I, g, x_k, d_k^\star)$, the field is trained using

$$\mathcal{L}_{\text{field}}(\theta) = \frac{1}{2}\left\| F_\theta(I, g, x_k)_{1:H-1} - d^\star_{k,1:H-1} \right\|_2^2 .$$

The first waypoint is excluded because it is fixed at the robot origin. The ESDF and energy computation do not appear inside this supervised SGD objective; they are used only when querying the oracle.

E. Energy-Based Dataset Aggregation

Training the field only around a fixed initialization would introduce distribution shift: after several learned refinement steps, the model may encounter trajectories that are substantially different from the initial training states. We therefore train GEAR using an iterative dataset aggregation procedure.

For every training condition, we first sample a structured trajectory x_0 from a goal-biased forward unicycle prior:

$$x_0 \sim q_{\text{prior}}(x \mid g).$$

Unlike Gaussian noise, this initialization provides a coarse, kinematically plausible trajectory that approximately points toward the goal.

At aggregation round k, we perform the following steps:

1) Query the geometric oracle once at the current trajectory x_k to obtain $d_k^\star$.
2) Add the state-direction tuple $(I, g, x_k, d_k^\star)$ to the replay dataset.
3) Train the VLA field on samples drawn from all previously aggregated rounds.
4) Apply the newly trained field to the current frontier to obtain the next model-induced trajectory state.

After K rounds, the replay distribution is

$$\mathcal{D}_K = \bigcup_{k=0}^{K-1} \left\{ (I_i, g_i, x_{i,k}, d^\star_{i,k}) \right\}_{i=1}^{N} .$$

During *aggregation only*, the learned update is protected by the same projection-aware energy backtracking used to validate an oracle step. Thus, the next training frontier is obtained from the learned field while preventing early inaccurate models from moving the replay distribution into high-energy or invalid regions. The oracle is then queried again only after this new frontier has been reached.

Aggregation continues until the trajectory updates converge or improvement on a fixed, clip-disjoint model-only validation set plateaus. After collection stops, the cached replay can optionally be used for additional SGD without querying the oracle again.

F. Model-Only Trajectory Refinement at Inference

The analytic energy guard used during aggregation is strictly a training-time mechanism. At inference, GEAR first constructs the same goal-biased trajectory prior $x^{(0)}$ and repeatedly applies the learned autonomous field:

$$x^{(\ell+1)} = \Pi_{\mathcal{K}}\left(x^{(\ell)} + \eta F_\theta(I, g, x^{(\ell)}) \right),$$
$$\ell = 0, \ldots, L-1.$$

We use a fixed refinement budget of $L = 12$ steps in our experiments. No ESDF query, trajectory-energy evaluation, negative-energy gradient, or energy-based acceptance test is performed during this rollout. The resulting policy therefore requires only the visual observation, goal specification, and lightweight kinematic projection at deployment.

V. ANALYSIS AND RESULTS

A. Experimental Setup

Baselines. We adopt the same three navigation baselines as VEGA [1]: OmniVLA [2], an omni-modal navigation VLA supporting language, image, and spatial goals; TIC-VLA [3], a think-in-control VLA for navigation in dynamic environments; and NaVILA [4], a vision-language-action model for legged robot navigation.

TABLE I

ISAAC-SIM NAVIGATION. VEGA BASELINES USE 50 TRIALS PER SCENARIO.

	Scenario 1			Scenario 2			Scenario 3		
Method	$S\uparrow$	$C\downarrow$	$Cl\uparrow$	$S\uparrow$	$C\downarrow$	$Cl\uparrow$	$S\uparrow$	$C\downarrow$	$Cl\uparrow$
OmniVLA	0.42	0.58	0.04	0.06	0.96	0.00	0.04	0.16	0.13
TIC-VLA	0.18	0.50	0.45	0.90	0.04	0.48	0.74	0.06	0.61
NaVILA	0.80	0.00	0.84	0.90	0.02	0.53	0.00	0.00	1.52
GEAR-Nav	0.92	0.06	0.78	0.84	0.10	0.46	0.86	0.12	0.71

S: success rate; C: collision rate; Cl: clearance (m).
Baselines: VEGA [1]. GEAR-Nav values supplied separately; run provenance and protocol comparability require verification.

GEAR-Nav's task-level evaluation will use the *model-only* rollout defined in Sec. IV-F, without the energy oracle or the training-time acceptance guard.

Evaluation protocol. Our comparison covers Isaac-Sim navigation and real-world robot navigation. The baseline values in Tables I and II are reproduced from VEGA [1]; they are not new measurements from our evaluation. The GEAR-Nav simulation values in Table I were supplied separately; their run provenance and comparability with the reference protocol remain to be verified. Real-world GEAR-Nav results are pending. In the reference real-world evaluation, policy outputs use a shared waypoint-tracking stack without additional path planning or obstacle avoidance. NaVILA's high-level commands are converted to local waypoints; its native LiDAR-conditioned obstacle-avoiding locomotion policy is not used [1].

Metrics. For the simulation and real-world comparisons, we retain the metrics and units reported in VEGA [1]:

- **Success rate** (S): fraction of successful goal-reaching trials. In the real-world evaluation, success requires reaching the goal without collision or safety intervention.
- **Collision rate** (C): fraction of trials with a collision; real-world safety interventions also count as collisions.
- **Obstacle clearance** (Cl): average minimum distance to obstacles over non-colliding trials, in meters. N/A denotes cases in which all trials collide.

Success and collision rates are reported on a $[0, 1]$ scale.

For the refinement analysis in Sec. V-D, we additionally consider goal progress, required-progress ratio, dense collision rate, and safe success under the robot-footprint margin. These geometric diagnostics are distinct from the trial-level metrics in the comparison tables.

B. Simulation Evaluation

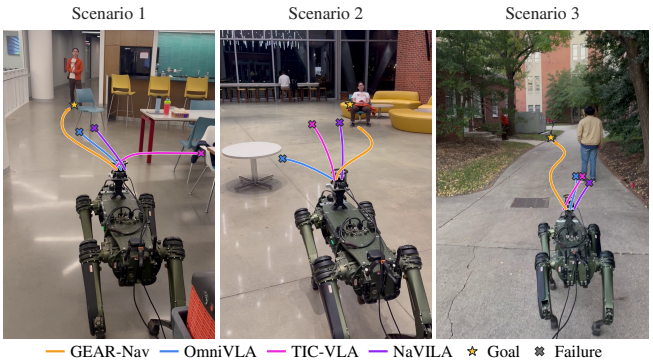
We adopt three Isaac-Sim scenarios containing static obstacles and moving pedestrians. The reference evaluation varies pedestrian motion onset using identical timing conditions across methods, with 50 trials per method and scenario [1]. Table I records the reported baseline results and the supplied GEAR-Nav values.

TABLE II

REAL-WORLD NAVIGATION (20 TRIALS PER METHOD AND SCENARIO).

	Scenario 1			Scenario 2			Scenario 3		
Method	$S\uparrow$	$C\downarrow$	$Cl\uparrow$	$S\uparrow$	$C\downarrow$	$Cl\uparrow$	$S\uparrow$	$C\downarrow$	$Cl\uparrow$
OmniVLA	0.15	0.85	0.35	0.00	0.65	0.32	0.00	0.50	0.44
TIC-VLA	0.00	1.00	N/A	0.10	0.85	0.37	0.00	1.00	N/A
NaVILA	0.00	1.00	N/A	0.00	0.80	0.35	0.00	0.45	0.31
GEAR-Nav	–	–	–	–	–	–	–	–	–

S: success rate; C: collision rate; Cl: clearance (m).
Baselines: VEGA [1]. "–": pending; N/A: all trials collide.



C. Real-World Robot Evaluation

Our real-world comparison covers three cluttered scenarios containing static and dynamic obstacles. Table II reproduces the baseline results for Scenarios 1–3 of VEGA's real-world evaluation, with 20 trials per method and scenario [1]. GEAR-Nav's quantitative results remain pending. Figure 4 illustrates schematic routes in these scenarios. GEAR-Nav routes summarize our recordings; the baseline overlays are adapted from the qualitative VEGA comparison [1].

The real-world experiments will test whether the geometric behavior distilled through energy-based aggregation transfers to scenes where no ESDF or trajectory-energy computation is available to the deployed policy. In particular, we will examine whether GEAR can remain goal-directed while modifying its trajectory around intervening static obstacles and newly observed dynamic obstacles.

D. Analysis of Learned Energy Refinement

We next isolate whether the learned field actually improves the structured trajectory prior and how closely it approaches privileged geometric optimization. We compare four rollout modes:

- **Prior**: the goal-biased unicycle initialization without learned refinement.
- **Model-only**: the deployable GEAR rollout using only the learned field and kinematic projection.
- **Guarded**: the learned field combined with privileged analytic-energy backtracking.
- **Oracle**: direct refinement using the analytic negative-energy-gradient oracle.

Guarded and Oracle use privileged scene geometry and are included only as diagnostic references. These comparisons

will evaluate the final trajectory after the fixed refinement budget.

The comparison between Prior and Model-only measures the improvement provided by the learned trajectory field itself. The difference between Model-only and Oracle characterizes the remaining gap between the visual policy and the privileged analytic optimizer, while the Guarded result indicates how much of this gap is caused by occasional inaccurate learned update directions or step magnitudes.

We additionally analyze performance as a function of refinement depth. A successful autonomous field should progressively transform the structured prior into trajectories with greater goal progress and lower collision rate, without requiring access to the geometric energy during inference.

APPENDIX

ACKNOWLEDGMENT

REFERENCES

[1] G. Seneviratne, Y. Abeysinghe, J. An, V. Shende, R. Kumar, and D. Manocha, "VEGA: Learning Navigation VLAs from In-the-Wild Egocentric Video with Geometric Trajectory Supervision," manuscript.

[2] N. Hirose, C. Glossop, D. Shah, and S. Levine, "OmniVLA: An Omni-Modal Vision-Language-Action Model for Robot Navigation," arXiv:2509.19480, 2025.

[3] Z. Huang, Y. Zhang, J. Liu, R. Song, C. Tang, and J. Ma, "TIC-VLA: A Think-in-Control Vision-Language-Action Model for Robot Navigation in Dynamic Environments," arXiv:2602.02459, 2026.

[4] A.-C. Cheng *et al.*, "NaVILA: Legged Robot Vision-Language-Action Model for Navigation," arXiv:2412.04453, 2025.